

Computer Science and Engineering

OnTime

Software System Design Specification (SSDS)

Version 3.0

Document Number: SSDS-002

Project Team Number: B1

Project Team Members: Anthony Lamelas (al8372), Chris Brasil (cdm9703), Uriel Olayinka (umo204), Edward Kang (emk9058)

REVIEW AND APPROVALS

Printed Name and Title	Function (Author, Reviewer, Approval)	Date	Signature
Anthony Lamelas, Web Developer	Author (Submit to Brightspace)	2/17/26	Anthony Lamelas
Edward Kang, Data Scientist	Author	2/17/26	Edward Kang
Uriel Olayinka, Data Scientist	Author, Reviewer	2/17/26	Uriel Olayinka
Chris Brasil, Web Developer	Editor, Approval	2/17/26	Chris Brasil

REVISION LEVEL

Date	Revision Number	Purpose
2/14/26	version 1.0	Initial Version
2/17/26	version 2.0	Final Version (Domain)
3/5/26	Version 3.0	Final Version (Req/Analysis)

TABLE OF CONTENTS

1. DOCUMENT PURPOSE.....	4	1.1
PURPOSE	4	2.
INTRODUCTION.....	4	
2.1 SCOPE.....	4	2.2
IDENTIFICATION	4	2.3
BOUNDS.....	5	2.4
OBJECTIVES	5	2.5
CONTEXT DIAGRAM	5	2.6
ADDITIONAL DESCRIPTIVE ITEMS	5	
3. GLOSSARY	6	4.
REFERENCE DOCUMENTS.....	6	5.
BUSINESS REQUIREMENTS.....	6	
5.1 TECHNOLOGY.....	6	5.2
ECONOMICS	6	5.3
REGULATORY AND LEGAL.....	6	5.4
MARKET CONSIDERATIONS	7	5.5 RISKS
AND ALTERNATIVES	7	5.6 HUMAN
RESOURCES AND TRAINING	7	
6. APPENDICES.....	7	
6.1 Schedule Tracking.....	7	
6.2 Defect Tracking.....	8	

1. DOCUMENT PURPOSE

1.1 Purpose

The purpose of this Software System Design Specification (SSDS) is to define the requirements, analysis, and design of the OnTime system. This document describes the system architecture, functional and non-functional requirements, system components, interfaces, and overall design decisions necessary to implement a subway delay prediction and route recommendation application.

This document serves as a technical reference to ensure consistency in system development, implementation, and maintenance.

This SSDS provides a structured description of:

- Functional requirements of the delay prediction and recommendation system
- Non-functional requirements such as performance, scalability, and reliability
- System architecture, including machine learning components, backend services, and frontend interface
- Data collection, preprocessing, model training, and deployment pipeline
- User interaction and system interfaces

These specifications will guide the development team in implementing a reliable, scalable, and user-friendly system.

Intended Audience

The intended audience of this SSDS includes:

- Development team members responsible for implementing the system
- Course instructor and advisor (Professor Strauss)
- Project stakeholders and evaluators
- Future developers and maintainers of the system
- System testers and deployment engineers

This document ensures that all stakeholders have a clear and consistent understanding of the system design and functionality.

2. INTRODUCTION

System Identification

The system to be produced is called **OnTime**, a machine learning–based subway delay prediction and route recommendation application.

OnTime will collect historical and live subway data, predict whether a train will be delayed, and recommend the best route to users based on delay likelihood and real-time transit conditions.

The system will consist of:

- Machine learning prediction model
- Backend prediction and recommendation API
- Web-based user interface
- Data ingestion and preprocessing pipeline
- Cloud-hosted infrastructure

Customer Problem Statement

New York City subway riders frequently experience delays that are unpredictable and disruptive. Currently, riders do not have access to tools that predict delays before they occur or provide proactive route recommendations based on delay probability.

This leads to:

- Missed classes and work
- Increased commuting stress
- Inefficient route selection
- Reduced confidence in public transit reliability

The OnTime system will address these problems by:

- Predicting whether a selected train is likely to be delayed
- Providing alternative route recommendations based on predictions
- Incorporating real-time and historical data to improve accuracy
- Allowing users to interact with predictions through a web application

System Application, Benefits, Objectives, and Goals

Application

OnTime is a web-based application designed for subway commuters, students, and workers who rely on public transit and need reliable trip planning tools.

Users will be able to:

- Select train lines, stations, and travel times
- View predicted delay probability
- Receive recommended routes based on delay predictions
- View real-time system conditions

- Save frequently used routes and preferences

Benefits

The key benefits of the system include:

- Improved commuter planning through predictive insights
- Reduced uncertainty and stress for transit users
- Increased reliability and efficiency in route selection
- Better utilization of public transportation infrastructure
- Insights into delay patterns for future analysis

Objectives

The primary objectives of the OnTime system are:

- Develop a machine learning model capable of predicting subway delays
- Integrate live transit data into prediction workflows
- Provide route recommendations based on predictions
- Deliver results through a responsive web application
- Deploy the system using a scalable cloud infrastructure

Goals

The system aims to:

- Improve subway commute reliability
- Enable proactive trip planning
- Provide real-time predictive insights
- Deliver accurate and fast predictions
- Ensure scalability and availability through cloud deployment

Consistency with System Proposal

This SSDS is fully consistent with the OnTime project proposal.

The proposal defines:

- Machine learning delay prediction model
- Real-time data integration
- Route recommendation system
- Web-based interface
- Cloud deployment using AWS

These elements are reflected in this system design specification.

Product Perspective

The OnTime system is a standalone web-based application that integrates with external data sources.

External Systems

The system will interface with:

- MTA Open Data Program (historical subway data)
- Live transit data sources (real-time subway information)
- AWS Cloud Infrastructure (EC2, S3, API Gateway, CloudWatch)

Internal Components

The system consists of the following components:

1. Data Collection Module
2. Data Preprocessing Module
3. Machine Learning Model
4. Prediction API
5. Recommendation Engine
6. Web Application Interface
7. Cloud Deployment Infrastructure

System Context

Users interact with the web application, which communicates with the backend API. The API retrieves live data, applies the machine learning model, and returns predictions and recommendations.

2.1 Scope

Functionality Included in the System

The OnTime system will include the following functionality:

Prediction Functionality

- Predict whether a train will be delayed
- Provide probability scores for delay likelihood

- Use machine learning trained on historical MTA data

Recommendation Functionality

- Recommend optimal routes based on predictions
- Avoid routes with high predicted delays
- Provide alternative travel options

User Interface Functionality

- Allow users to select:
 - Train lines
 - Stations
 - Travel times
- Display:
 - Delay predictions
 - Recommended routes
 - Prediction confidence
- Allow users to save frequently used routes and preferences

Data Processing Functionality

- Collect historical MTA data
- Clean and preprocess data
- Perform feature engineering
- Train and evaluate machine learning models

Backend Functionality

- Provide a REST API for prediction requests
- Handle real-time data ingestion
- Process user requests and return predictions

Deployment Functionality

- Cloud deployment using AWS
- Containerization using Docker
- Model storage using S3
- Monitoring using CloudWatch

Functionality NOT Included in the System

The following functionality is outside the scope of this project:

- Mobile application development (web app only)
- Direct integration with official MTA internal systems
- Real-time GPS tracking of trains
- Ticket purchasing functionality
- User authentication with external identity providers
- Guarantee of 100% prediction accuracy
- Full replacement of official transit planning systems

2.2 Identification

SSDS-Domain Document, Group B1, Revision version 3

2.3 Bounds

OnTime is a web-application that helps users plan their subway trips by generating delay predictions using historical MTA delay data as well as live data. Through the use of these datasets, OnTime will present a predicted delay risk and route recommendation to the user.

In Scope

- Trip Planning Support: Allows users to select train lines, stations, and times and view predicted delays in a clear format
- Delay prediction: Uses machine learning trained on preprocessed historical MTA data to generate delay predictions
- Live data integration: Supports live data ingestion for real-time predictions, but only pulls live data requested by the user to avoid performance degradation
- Recommendation behavior: Produces route recommendations using model output and real-time conditions
- User profiles (lightweight): Let users save frequently used routes so those routes can be prioritized for scraping
- Deployment boundary: Cloud deployment on AWS with an inference API and monitoring as described in the proposal

Out of Scope

- Does not guarantee elimination of delays; it provides predictions and recommendations only
- Does not provide official MTA dispatching/operations control features
- Does not continuously scrape all subway live data at all times; live ingestion is on-demand based on user requests

2.4 Objectives

Project Priority

OnTime is a high-impact, commuter-facing project intended to improve trip planning reliability in NYC by helping riders plan around delays rather than react to them, addressing frequent and unevenly distributed subway delays that affect students and workers.

Type of Delivery/Life Cycle

OnTime will be delivered incrementally, aligned with the planned SSDS deliverables and implementation timeline.

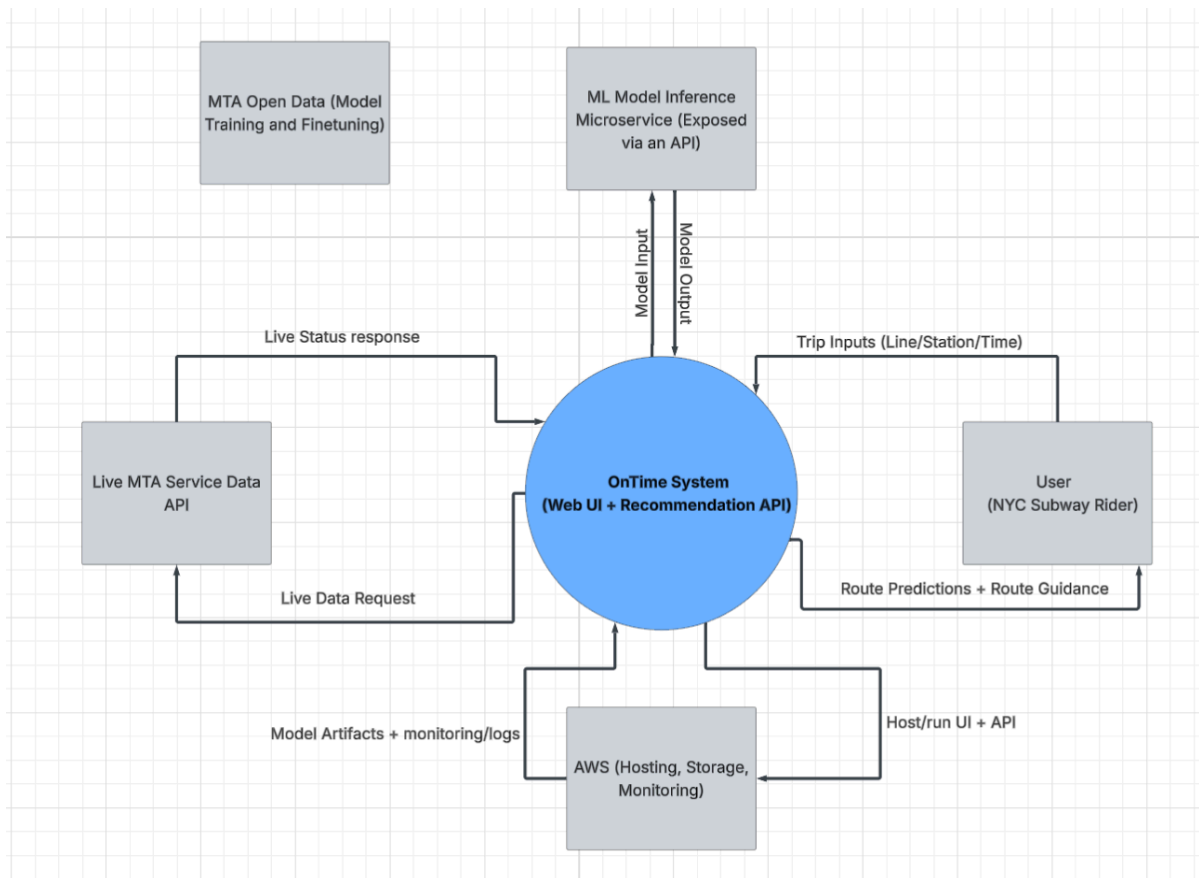
Milestone Dates

Project Proposal: February 3, 2026
Project Information Description: February 26, 2026
SSDS Domain: February 19, 2026
SSDS Requirements/Analysis: March 5, 2026
SSDS Design: May 5, 2026

System Objectives

- Provide a web UI where users can select train lines, stations with time windows, and receive predicted delay information
- Train and validate machine learning models using features from MTA delay datasets to support prediction
- Integrate live data ingestion in a performance-safe way by fetching only when the user requests data in real time
- Set up a separate microservice for model inference
- Deploy on AWS with containerization and monitoring
- Data Collection

2.5 Context Diagram



2.6 Additional Descriptive Items

Product Functions

- Ingests and preprocesses historical MTA delay data for ML training
- Trains/evaluates models and generates delay predictions
- Pulls live data only when users request it to avoid performance issues
- Produces route recommendations that account for real-time service status
- Provides a web UI for selecting line/station/time and viewing results
- Supports basic user preferences (saved routes) and serves predictions via an API

User Characteristics

- Intended users are NYC subway riders (especially students/workers) who want quick, practical delay guidance
- Users are assumed to know their usual lines/stations but do not need technical expertise

- Repeat users benefit from saved preferences/routes

Constraints

- Data constraints: limited to what MTA historical/live sources provide
- Performance constraints: live scraping is on-demand, not continuous
- Interface constraints: predictions delivered through a REST API for the web app
- Deployment constraints: hosted on AWS with containerized deployment and monitoring

Assumptions and Dependencies

- Historical MTA data is available and sufficient to train useful models
- Live data access remains available; if degraded, results may rely more on historical signals
- AWS services and deployment pipeline remain stable for hosting and monitoring
- Service-status detection is accurate enough to avoid recommending unavailable routes

Requirements Subsets

- MVP: historical data pipeline + trained model + web UI predictions
- Next: bounded live-data integration + service-aware recommendations
- Deployment: API + AWS hosting/monitoring
- Future: personalization (saved routes/preferences) and performance refinements

3. GLOSSARY

- **Overfitting:** a concern in machine learning where the model memorizes the training data, reducing its quality when viewing unseen information
- **Feature transformation:** the practice of applying mathematical transformations to all features of a dataset, often used to make the models pick up more and less complex relationships
- **Hyperparameter tuning:** the practice of changing certain model parameters to improve performance
- **Web scraping:** the gathering of information from the internet using code to simulate human interaction
- **REST API (Representational State Transfer):** The architectural style and set of protocols used by the Web Application Interface to communicate with the Prediction API
- **Microservice:** A specialized, independent service, such as the ML Model Inference service, that performs a single function within the larger OnTime system
- **Containerization:** The process of packaging the application and its dependencies into a "container" (using tools like Docker) to ensure it runs consistently across different computing environments
- **Inference:** The process of using a trained machine learning model to compute a delay prediction for new, real-time trip queries
- **Data Ingestion:** The process of fetching and importing historical or live subway data from external sources into the OnTime system
- **Preprocessing:** The cleaning and preparation of raw data before it is used for model training or inference

4. REFERENCE DOCUMENTS

B1, OnTime Project Proposal, Version 3, 5/5/26

5. BUSINESS REQUIREMENTS

5.1 Technology

Demand for Reliable Urban Mobility:

Millions of people ride the subway every single day, which makes delays and other subway disturbances cost millions of dollars. Especially with a growing population post-pandemic, it is imperative that the subway runs as efficiently as possible. We intend to improve this experience by providing tailored transit recommendations for our users.

Large-Scale Open Data Analysis

The MTA does an excellent job of open sourcing their data, however, there is a lack of in-depth research and analysis that explores such data. In order to develop our application, we will have to perform in-depth, large-scale analysis and model training using years of subway data. This will not only allow our app to be high-quality, but will also allow the general public to learn more, as we intend to open-source our findings.

5.2 Economics

Economic Impact on a Train-Dependent Population

In New York City, the majority of people who work in the city rely on the trains to travel to and from work. This is especially true for lower-income communities that cannot afford other forms of transportation, such as personally-owned vehicles or ride-share apps.

Scalability to Other Transit Systems

If our system is able to effectively understand, predict, and model subway routes, it could be adapted to other cities, increasing long-term economic and institutional value. This would also greatly increase the revenue and impact that our app is able to generate.

5.3 Regulatory and Legal

MTA Open Data Terms of Service

The project relies on the MTA Open Data program. This project must comply with their terms of agreement in order to ensure continued access to subway data.

Data Privacy

Although the system focuses on train data, our system will include a feature that allows users to save certain preferences and routes. This creates the need for data protection measures to ensure that user information is handled securely.

Terms of Use and Liability

Since this system will provide a prediction on whether or not a subway will be late, it is important to include a disclaimer that the project is for informational purposes. This will protect the team if a user's schedule is put off due to an incorrect prediction.

version 3.0 6 2/19/26

SYSTEMS ENGINEERING PROCESS SOFTWARE SYSTEM DESIGN SPECIFICATION SSDS-001

5.4 Market Considerations

Commuter Productivity

NYC subway delays cause late arrivals to work/class. There is a high demand for tools that offer predictive planning rather than just reactive alerts. By accounting for various issues, the project can minimize stress that reduces public confidence in public transit reliability.

Digital Upgrade of Infrastructure

Our system is an opportunity to provide digital upgrades to aging infrastructure without requiring billions of dollars of physical overhauls.

5.5 Risks and Alternatives

Risk Type	Description	Probability	How Discovered	Responsible Party	Status	Mitigation Plan
Business	Low usage rate due to lack of trust in predictions	Medium	Testing	All	Open	Use years of historical MTA data to prove the accuracy of models
Operational	Low performance of web app due to excessive real-time data collection	High	Stress testing	Full-stack developers	Open	Only scrape data requested by the user and prioritize frequently used routes
Technology	Model overfitting; failing to generalize to real-world data	Medium	Model evaluation results	Data scientists	Open	Feature transformations, hyperparameter tuning, and testing multiple architectures
Economic	Deployment costs	Low	Deployment service billing	All	Open	Use cheaper deployment services, and use Docker containers for resource efficiency

5.6 Human Resources and Training

Team Roles:

- Data Scientists: responsible for feature engineering and model development
- Full-Stack Developers: responsible for web UI and RESTful API

Required Training

While no formal training is required, the web developers must be competent in full-stack development as well as cloud deployment, while our data scientists must be competent in data collection, data preprocessing, model training, and model deployment.

6. USER REQUIREMENTS (DESCRIPTIVE - FUNCTIONAL, AND NON-FUNCTIONAL REQUIREMENTS)

6.1 Functional Descriptive Detailed Requirements

OnTime system's functional requirements define the key features and behaviors it must support for end-users and maintainers. The system shall fulfill the following functional requirements:

1. **User Trip Query Input:** The system shall provide a web-based user interface through which a user can input a trip query by selecting a subway line, an origin station, a destination station, and a desired travel time (departure or arrival window). This input flow must be intuitive, allowing users to easily specify their NYC subway trip parameters (with optional station lookup or auto-complete, via Google Maps API to assist in finding valid station names).
2. **Delay Prediction Computation:** Upon submission of a trip query, the system shall compute a delay prediction for the specified route and time. The prediction is generated by running an inference on a trained machine learning model that has learned from historical MTA subway data. The model will output the likelihood of a delay occurring on the chosen route at the given time, leveraging patterns from historical trends.
3. **Live Data Integration:** The system shall enhance its predictions with on-demand live data to improve accuracy and responsiveness to current conditions. For example, if a live data source indicates incidents, outages, or other relevant events at query time, the backend shall incorporate this information before finalizing the delay prediction. Live data is fetched on-demand rather than continuously, to ensure fresh information is used without incurring unnecessary data calls when the system is idle.
4. **Result Output – Delay Likelihood:** The system shall return a prediction result to the user that includes both the delay likelihood and the estimated impact of any potential delay. The delay likelihood may be expressed as a probability or a qualitative level. The impact is an estimate of how the delay would affect the trip if it occurs. For example, an additional travel time in minutes or a categorical description such as minor, moderate, or severe delay. The result is presented clearly through the web UI so that the user can understand the risk associated with their planned route.
5. **Route Recommendation Behavior:** If the system predicts a significant delay on the user's chosen route, it shall proactively offer a route recommendation to mitigate the impact. This means the system will identify an alternative route or travel option that could be faster or more reliable given the predicted delay. The recommendation might involve suggesting a different train line or a different transfer path between the origin and destination. Where applicable, the system may utilize an external transit like the Google Maps API to find and validate alternate routes. The recommended route, along with its expected travel time and any comparative benefit, shall be displayed to the user. If no better alternative is available the system will simply inform the user of the predicted delay without an alternate suggestion.
6. **Data Pipeline and Model Maintenance:** The system shall include an internal data pipeline that regularly aggregates and processes new data. This involves fetching periodic updates of MTA subway performance data. Data science team members can retrain and improve the model as new data arrives, and the updated model shall be deployed to the prediction service without disrupting the user-facing application.

6.2 Non-Functional Descriptive Detailed Requirements

The **OnTime** system must also satisfy several non-functional requirements that stipulate performance standards, quality attributes, and operational constraints. These requirements ensure the system is reliable and maintainable:

1. **Performance:** The system shall deliver predictions with low latency to support real-time usage. From the moment a user submits a trip query, the target response time for computing the delay prediction and displaying results should be on the order of a few seconds or less. This performance requirement includes the time to fetch on-demand live data and run the ML inference. The design will avoid long-running computations on the main request path and will fetch external data efficiently so that the user interface remains responsive.
2. **Scalability:** The system shall be scalable to handle increasing numbers of users and growing data volumes without degradation of performance. The architecture should support horizontal scaling of the web front-end and backend API servers to serve more concurrent users as demand grows. Similarly, the data processing and model inference components should be able to scale to accommodate larger datasets and more frequent queries. The design will ensure that adding resources can linearly improve throughput, allowing OnTime to serve the NYC commuter population as it grows.
3. **Availability and Reliability:** The system shall be highly available, providing reliable access to predictions with minimal downtime. The deployment on AWS should leverage best practices for high availability. For example, running multiple instances of critical services across different availability zones, using load balancers to distribute traffic, and employing automated health checks and failover mechanisms. In the event of a server failure or network issue, the system should continue to operate rather than becoming completely inaccessible. The target availability for the OnTime service might be, for example, 99% uptime or higher, recognizing that commuters may rely on it at various hours.
4. **Modular Design and Maintainability:** The system shall be designed in a modular, component-based fashion to separate concerns and improve maintainability. The architecture cleanly differentiates the front-end user interface, the backend application logic, and the data pipeline components. This modular design means each component can be developed and modified independently. Clear interfaces will be defined between these modules. As a result, updates such as swapping in a new machine learning model, changing the data source, or updating the UI can be done with minimal impact on other parts of the system.
5. **Data Freshness:** The usefulness of delay predictions depends on data freshness, so the system shall ensure that both historical and live data used for predictions are up-to-date. For the historical data, the system will regularly ingest new MTA data and update the training dataset. The machine learning model should be retrained on a scheduled basis (up for discussion) so that its predictions reflect the latest transit

patterns and seasonal trends. For live data, as noted, the system will retrieve current information at query time to incorporate any recent developments.

6. **Fault Tolerance (Live Data Degradation):** The system shall be fault-tolerant with respect to its external data sources and internal components. In particular, if live data feeds are unavailable, slow, or returning errors at the time of a query, the OnTime system should still gracefully provide a prediction using the historical-data model alone. The application will be designed to handle such exceptions without crashing or undue delay. For example, by applying timeouts to external API calls and falling back to the default behavior. Users may be notified in a subtle way if a live data source is temporarily unavailable, but the core functionality will remain operational.

7. SYSTEM ARCHITECTURE

The system follows a modular, service-oriented architecture that separates user-facing functionality, business logic, machine learning workflows, and persistent storage. This structure supports maintainability, scalability, and reuse of proven components while keeping each module focused on a clear responsibility.

High Level Architecture Modules

Client Interface Layer

- Provides all end-user interactions (e.g., web dashboard, admin pages, data upload screens, and result visualizations).
- This layer handles input validation, presentation logic, and API communication.
- Application/API Layer
- Implements core application workflows and exposes system functionality through REST endpoints.
- Responsibilities include request routing, authentication/authorization, orchestration of ML inference/training requests, and error handling.

Machine Learning Service Layer

- Hosts model-related functions such as preprocessing, feature engineering, inference, model retraining, and evaluation.
- This module is isolated from the UI so models can be updated independently from client-facing code.

Data Management Layer

- Manages structured and unstructured data through databases/object storage.
- Includes dataset versioning, metadata tracking, logging, and model artifact storage.

Infrastructure & Integration Layer

- Includes deployment/runtime services such as containerization, monitoring, logging, configuration management, and CI/CD.
- Also handles external integrations (if applicable), such as third-party APIs or cloud storage.

Functional Distribution Across Modules

System functions are distributed to maximize cohesion and minimize coupling:

- User interaction and visualization are handled by the Client Interface Layer.
- Business rules, access control, and workflow coordination are handled by the Application/API Layer.
- Prediction and model lifecycle operations are handled by the Machine Learning Service Layer.
- Persistent storage and retrieval are handled by the Data Management Layer.
- Operational reliability and external connectivity are handled by the Infrastructure & Integration Layer.
- This separation allows independent development/testing of modules and reduces risk when introducing new features.

Reused Architectural Components

To reduce development effort and improve reliability, the architecture reuses established components:

- Authentication/authorization middleware reused across all protected API endpoints.
- Standard API controller/service pattern reused for each functional domain.
- Common preprocessing pipeline reused between training and inference to ensure consistency.
- Shared logging and monitoring stack reused across application and ML services.
- Reusable database access layer/ORM patterns reused for all persistent entities.
- Containerized deployment templates reused for local development and production environments.

Architectural Rationale

- Supports future scaling (independent scaling of API and ML workloads),
- Isolates high-change areas (model logic vs. UI),
- Simplifies testing (module-level unit/integration boundaries),
- Improves maintainability through clear ownership and interfaces,
- Enables safe reuse of proven cross-cutting components.

8. DETAILED SYSTEM REQUIREMENTS - USE CASES

8.1 Requirement Use Cases

User-facing

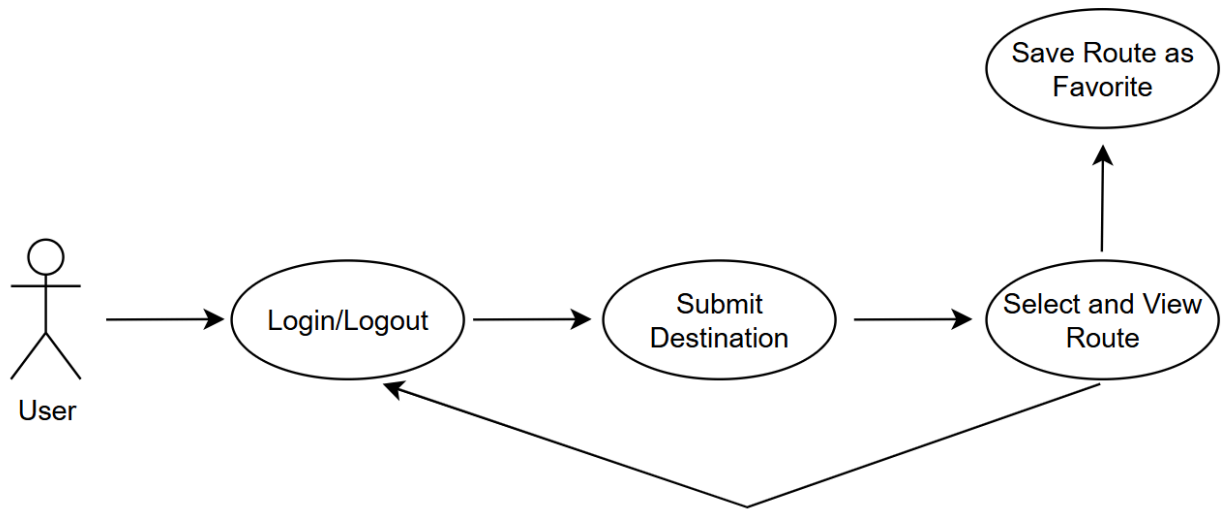
- Login/Logout
- Submit Destination
- Select Route from Recommendations
- Save Route as Favorite

Admin / monitoring

- Monitor System Health
- Review Live Data Source Status

- Retrain and Deploy Prediction Model
- Review Prediction Logs / Performance Metrics
- Handle Service Failure / Fallback Mode

8.1.1 Use Case Diagrams



8.1.2 Use Case Descriptions

1. Login/Logout

Description	The system allows a registered user to prove identity with email and password, after which the client holds credentials needed for protected actions (e.g., saved routes). Logout revokes the active session so subsequent requests are treated as unauthenticated.
Pre-Conditions	For login: a row exists in users with a bcrypt hashed_password. For register: email not already used. System (FastAPI, DB) is up.
Basic or Normal Flows	User opens login view, switches to register, submits email/password → POST /api/users/ → on success, client automatically calls POST /api/auth/login and sets logged-in state. User submits email/password → POST /api/auth/login → on success, client sets local logged-in state (token response is not stored in

	code as written). User clicks Logout in SidebarNav → onLogout sets isLoggedIn to false.
Alternative Flows	Invalid credentials → API 400 with Incorrect email or password. Register duplicate → 400 from users.py. Client shows error message from detail when possible (LoginPanel.jsx).
Post Conditions	Register/login success: client “logged in” flag is true. Logout: flag is false.
Special Requirements	Passwords are stored with bcrypt (get_password_hash / verify_password).
Extension Points	Persist JWT (e.g. localStorage) and send Authorization: Bearer on protected routes; server-side token blacklist; OAuth/MFA.

2. Submit Destination

Description	Allows the user to input trip parameters and request a route recommendation.
Pre-Conditions	stations.json and subway_graph.json loadable; for success, valid coordinates and a routable path (otherwise 404/exception per server).
Basic or Normal Flows	User fixes origin and destination → client validates presence indirectly (effect only runs when both set) → POST /api/subway/plan → UI receives origin_station, dest_station, route, travel_time (including line_departures, live flag) → TripPlanner shows station chips, optional line rows, and TravelCard breakdown.
Alternative Flows	No path / error: fetch failure sets plan to { route: { found: false } } (App.jsx); UI shows “No subway route found...” when !plan.route.found. Line preference: user selects a line; handleLineSelect updates selectedLine and re-fetches with preferred_line.
Post Conditions	Either a found plan with stations and times, or not found / error state as above.
Special Requirements	One plan per successful response; walk + wait + ride summary in TravelCard.
Extension Points	Integrate ml_service POST /predict for a delay

	risk score; return multiple path alternatives.
--	--

3. Select Route from Recommendations

Description	Allows the user to select one of the recommended routes for guidance and trip planning.
Pre-Conditions	plan is non-null; plan.route.found is true; line_departures populated as returned by the API.
Basic or Normal Flows	User reviews per-line departures at origin → selects line → new plan with preference → TravelCard and map update.
Alternative Flows	No line selected: Dijkstra may use the fastest overall line at origin. Map geometry: if track GeoJSON is missing, map may fall back to simpler geometry (see SubwayMap / routeSubwayGeoJSON).
Post Conditions	selectedLine reflects user choice or null; UI shows updated timing/breakdown.
Special Requirements	UI must allow easy switching between routes
Extension Points	Multiple ranked routes; in-trip notifications.

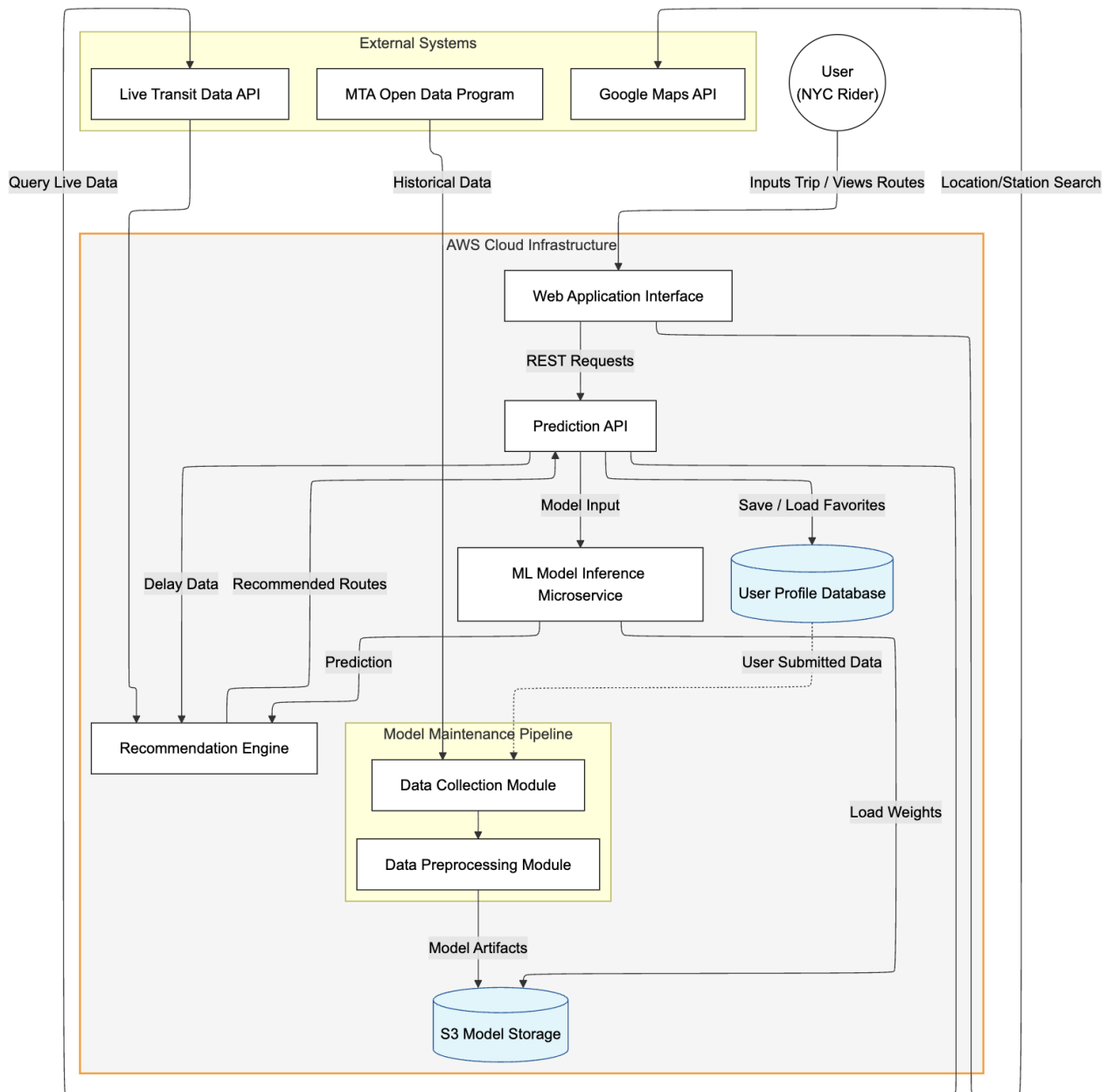
4. Save Route as a Favorite

Description	Allows the user to save a selected route for future quick access.
Pre-Conditions	User (sidebar shows Favorites when “logged in”; favorites data is not bound to users.id in the API)
Basic or Normal Flows	User opens Favorites view → may add or click a saved item → setOrigin / setDestination in App.jsx → returns to planner.
Alternative Flows	Route already saved - system notifies user Database error - the system displays an error message

Post Conditions	Map/planner state updates for the current session.
Special Requirements	Saved routes must persist across sessions
Extension Points	Add favorites table and authenticated CRUD; geocode text; save from current plan.

9. SYSTEM MODEL - UML

UML Component Diagram



9.1 Static Class Diagrams

1. Candidate Classes (list)

Entity Classes (data objects)

- User
- Route
- Station
- Trip

- DelayRecord
- TransitData
- PredictionResult

Interface / Boundary Classes

- WebApplicationInterface
- PredictionAPI
- LiveTransitDataAPI
- GoogleMapsAPI
- MTAOpenDataService

Control / Service Classes

- RecommendationEngine
- MLModelInferenceService
- DataCollectionModule
- DataPreprocessingModule

2. Class Diagram (without Interaction)

Main Data relationships

User

- user_id
- favorite_routes
- favorite_stations

Route

- route_id
- route_name

Station

- station_id
- station_name

Trip

- trip_id
- route_id
- direction

DelayRecord

- delay_seconds
- station_id
- route_id
- timestamp

PredictionResult

- predicted_delay
- route_id
- station_id
- confidence

System Service Classes

PredictionAPI

- getPrediction()
- getRoutes()
- getStations()

MLModelInferenceService

- loadModel()
- runInference()

RecommendationEngine

- recommendRoute()
- validateRoute()

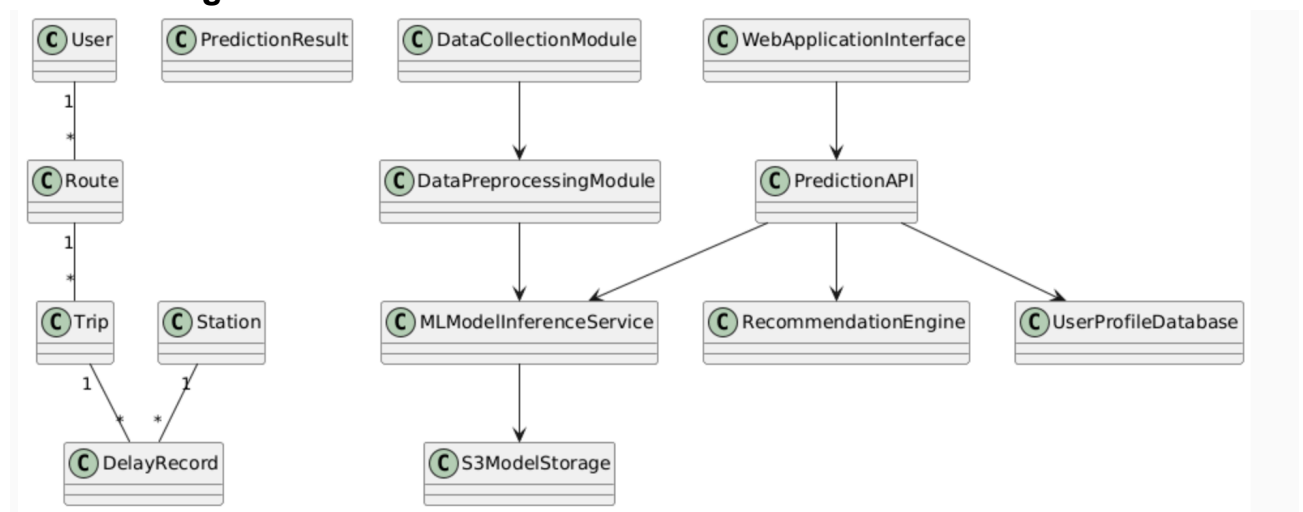
DataCollectionModule

- fetchHistoricalData()

DataPreprocessingModule

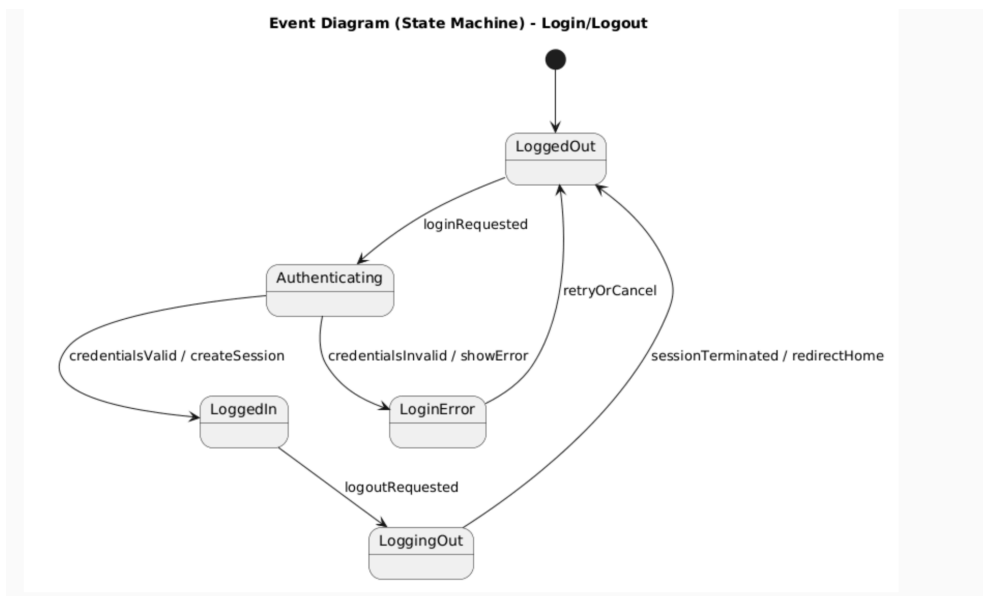
- cleanData()
- buildFeatures()

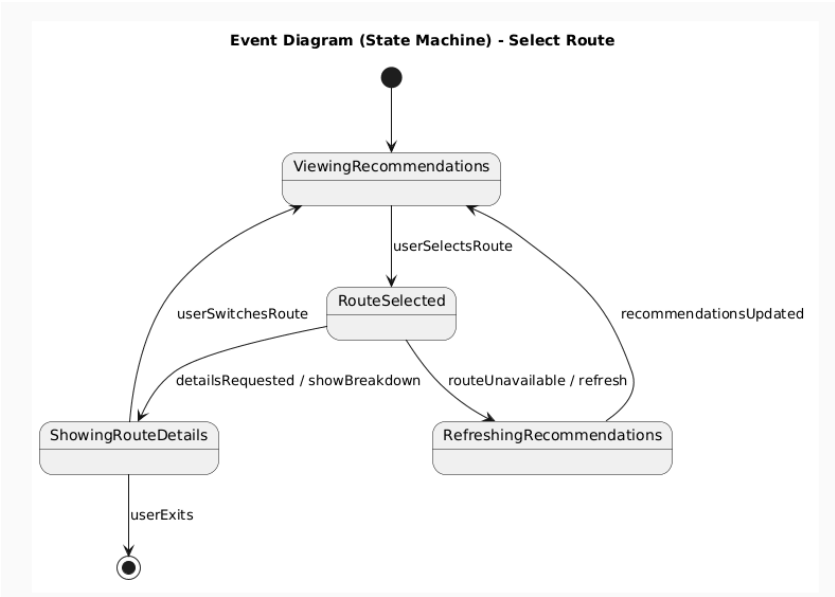
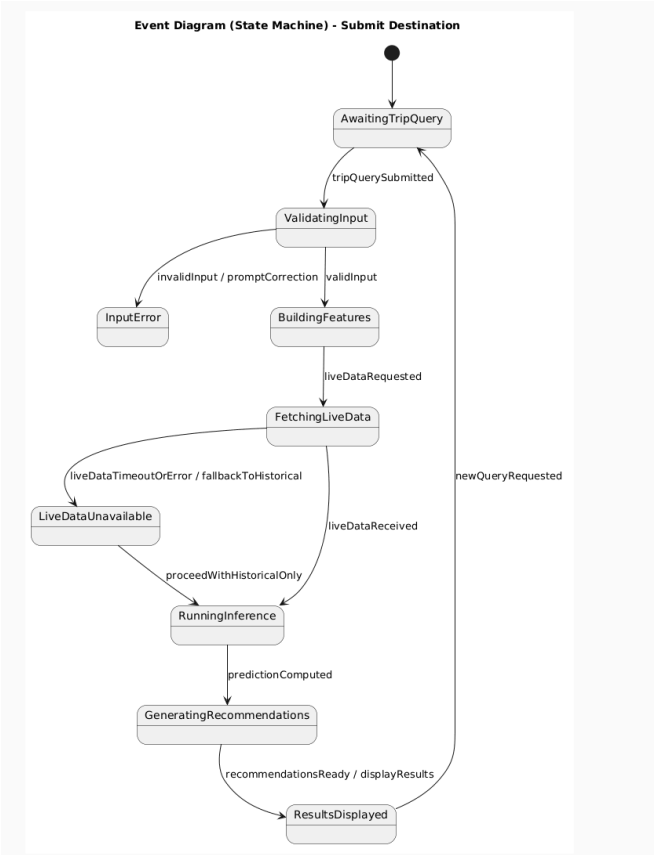
3. Class Diagram with interaction

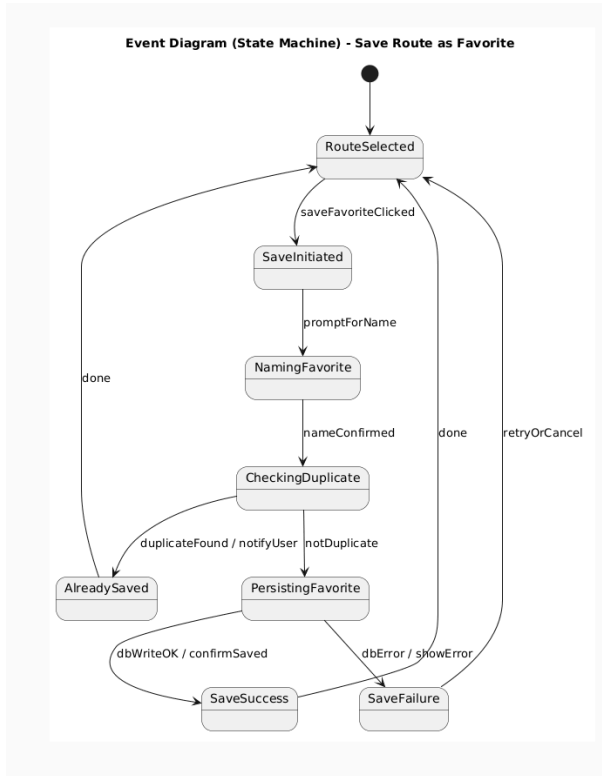


9.2 Dynamic - Behavior Models

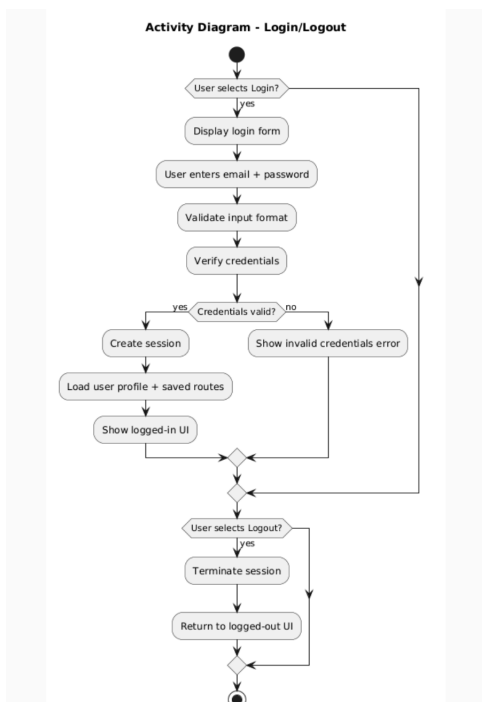
Event Diagrams

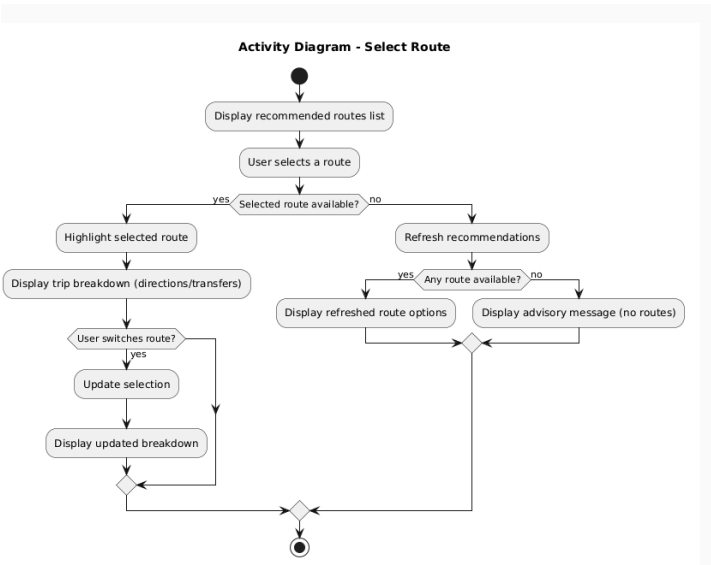
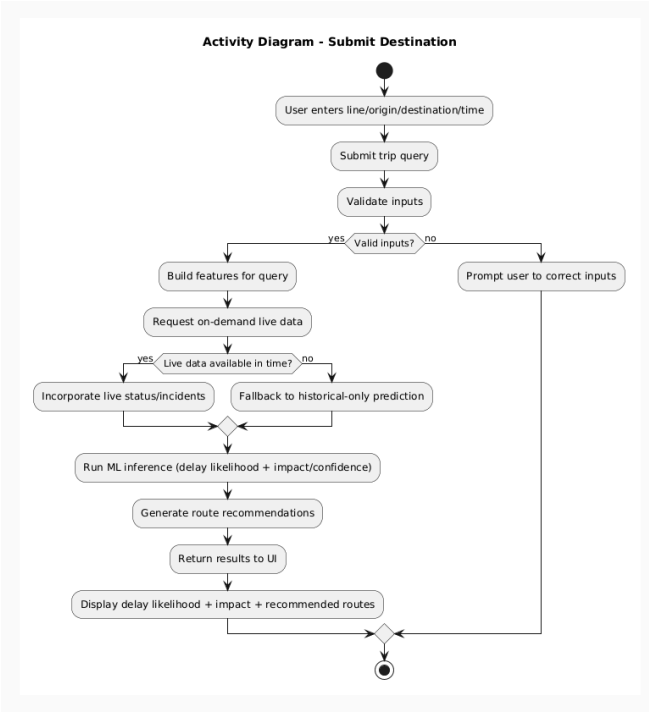


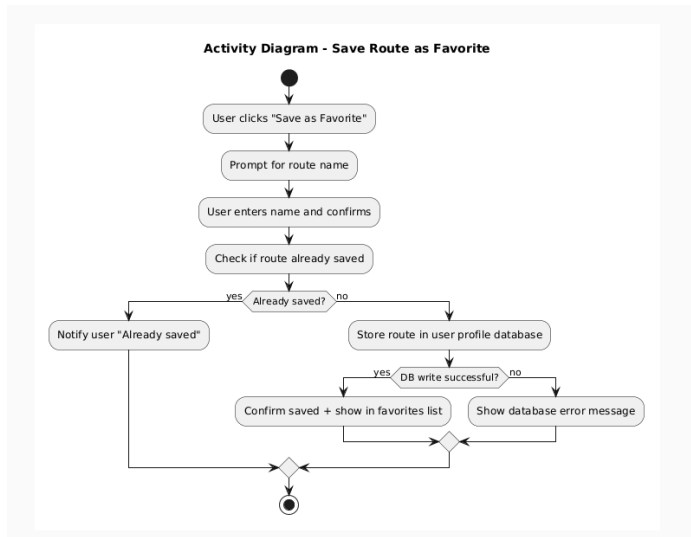




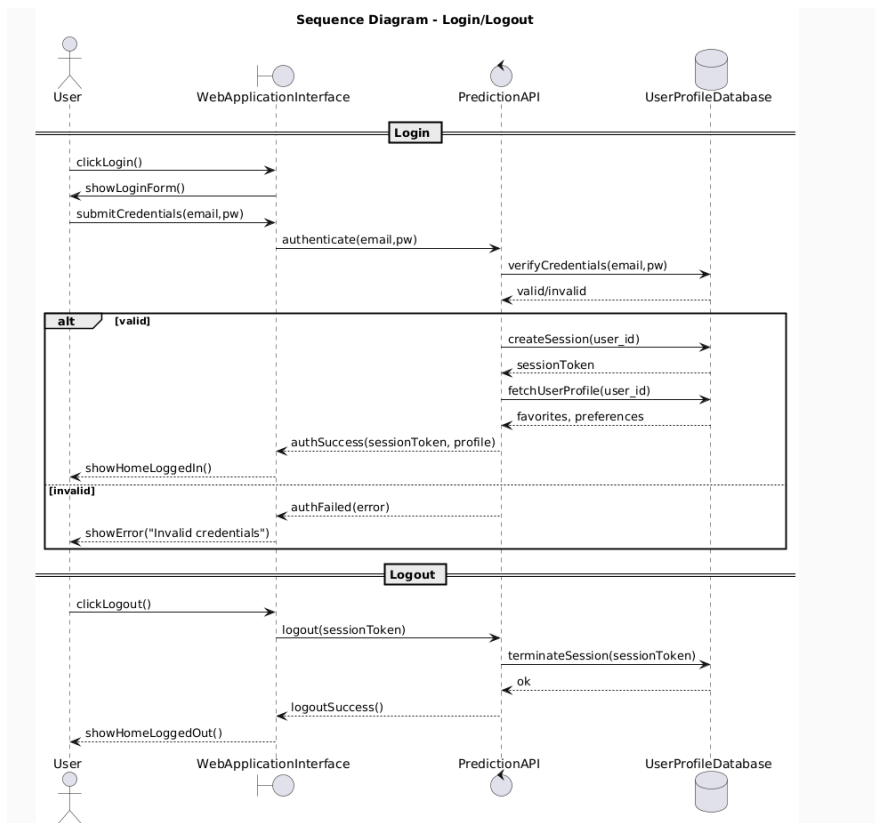
Activity Diagrams



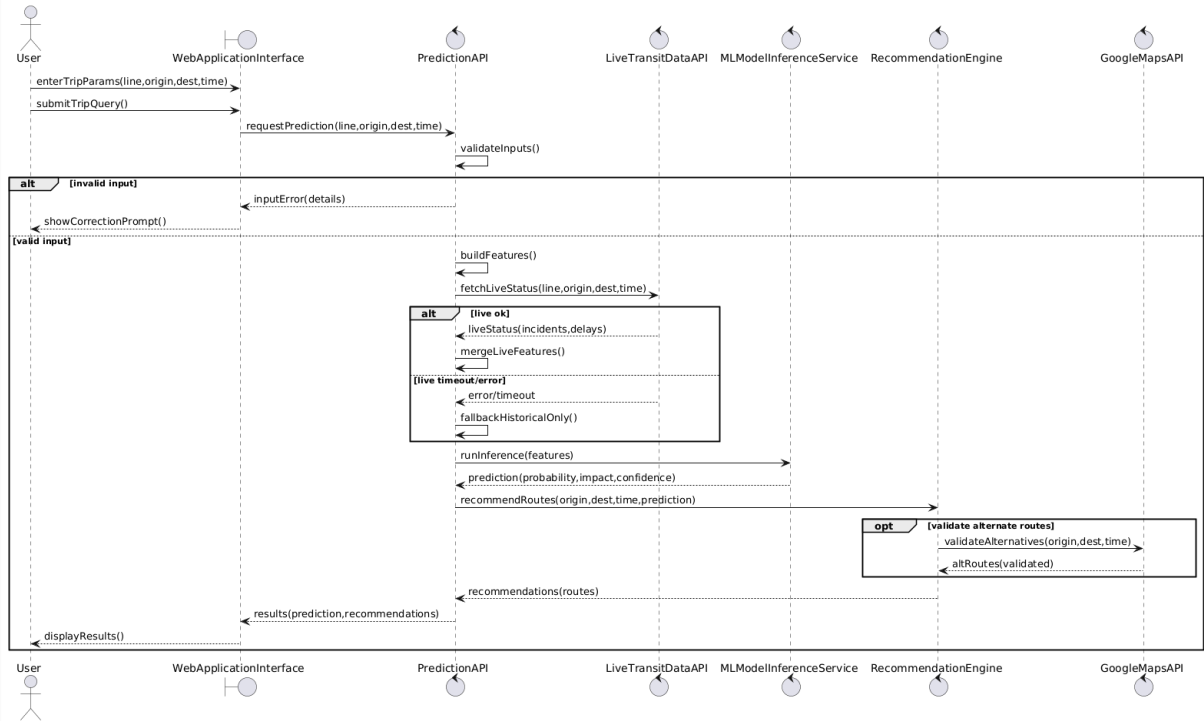




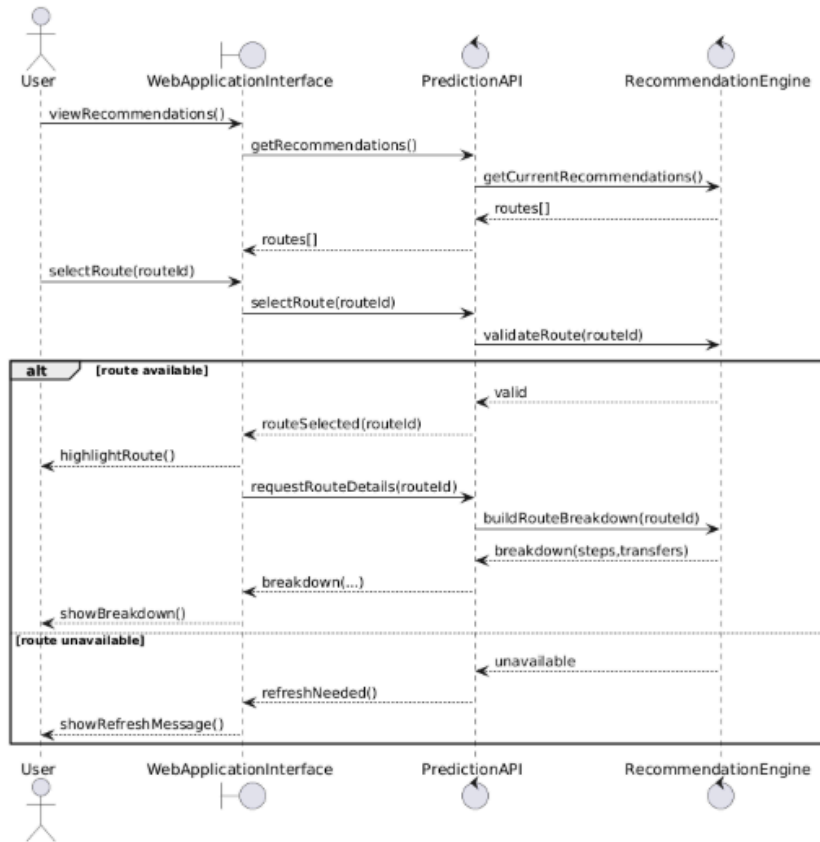
Sequence Diagrams

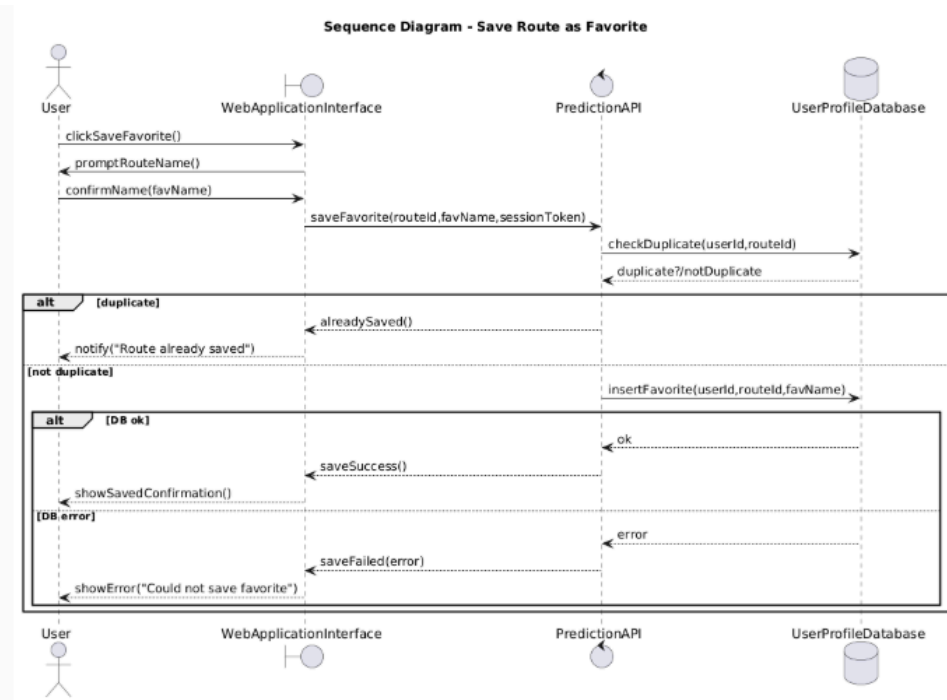


Sequence Diagram - Submit Destination



Sequence Diagram - Select Route





10 System-Wide Design Decisions

10.1 Software Component Architectural Design

10 Appendices

10.1 Schedule Tracking

Hours

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
Project Code Initial	All Members	3 Hours	2 Hours	1
SSDS - Requirements/ Analysis	All Members	3 Hours	3 Hours	0

Cumulative

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference

version 3.0 2/19/26

SYSTEMS ENGINEERING PROCESS SOFTWARE SYSTEM DESIGN SPECIFICATION SSDS-001

Worked on Initial Code and SSDS-Requirements	Anthony Lamelas	5	5	0
Worked on Initial Code and	Uriel Olayinka	5	5	0

SSDS-Requirements				
Worked on SSDS-Requirements	Chris Brasil	4	4	0
Worked on Initial Code and SSDS-Requirements	Edward Kang	5	5	3
Created Separate Microservice and CI/CID	Anthony Lamelas	2	2	0

10.2 Defect Tracking

Counts

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Requirements	Anthony Lamelas	0.5 hours (grammar/spell)	0.5	0
SSDS - Requirements	Uriel Olayinka	0.5 hours (grammar/spell)	0.5	0
SSDS - Requirements	Edward Kang	0.5 hours (grammar/spell)	0.5	0

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
SSDS - Requirements	Chris Brasil	0.5 hours (grammar/spell)	0.5	0
SSDS - Requirements	We did indeed have a lot of formatting/grammar errors, but we also misunderstood	2	4	2

	d some sections and had to redo them.			

version 3.0 14 2/19/26

SYSTEMS ENGINEERING PROCESS SOFTWARE SYSTEM DESIGN SPECIFICATION SSDS-001

Cumulative

Artifact or Deliverable	Who (individual or Team)	Estimated	Actual	Difference
Cumulative - Summary	Anthony Lamelas	0.5	1	0.5
Cumulative - Summary	Edward Kang	0.5	1.5	1
Cumulative - Summary	Uriel Olayinka	0.5	0.5	0
Cumulative - Summary	Chris Brasil	0.5	1	0.5
Cumulative - Summary	Summary for the entire Team	2	4	2

